

บทที่ 3

ไฟร์วอลล์

ไฟร์วอลล์ คือ เครื่องมือที่ใช้ป้องกัน และ ควบคุมการทำงานของแพ็กเก็ตข้อมูลที่ผ่านมาเข้าออก ภายในเครือข่าย

3.1 ประเภทของไฟร์วอลล์

ชนิดของไฟร์วอลล์แบ่งตามเทคโนโลยีที่ใช้ในการตรวจสอบ และควบคุมแบ่งได้เป็น 3 ประเภท ดังนี้คือ

1. แพ็กเก็ตฟิลเตอร์ริง (Packet Filtering)
2. พร็อกซีเซอร์วิส (Proxy Service)
3. สเตตฟูลอินสเปกชัน (Stateful Inspection)

3.1.1 แพ็กเก็ตฟิลเตอร์ริง (Packet Filtering)

แพ็กเก็ตฟิลเตอร์ริง (Packet Filtering) คือ เราเตอร์ หรือ ซอร์ฟแวร์ ที่ทำหน้าที่หาเส้นทางส่งต่อ (route) อย่างมีเงื่อนไข โดยจะพิจารณาจากข้อมูลส่วนที่อยู่ในเฮดเดอร์ (header) ของแพ็กเก็ตที่ผ่านมา เข้ามา เทียบกับกฎ (rules) ที่กำหนดไว้ และตัดสินใจว่าจะทิ้ง (drop) แพ็กเก็ตนั้นไปหรือว่าจะยอม (accept) ให้แพ็กเก็ตนั้นผ่านไปได้ ในการพิจารณาเฮดเดอร์ แพ็กเก็ตฟิลเตอร์ริง จะตรวจสอบในระดับของอินเทอร์เน็ตเลเยอร์ (Internet Layer) และทรานสปอร์ตเลเยอร์ (Transport Layer) ในอินเทอร์เน็ตโมเดล ซึ่งในอินเทอร์เน็ตเลเยอร์จะมีแอตทริบิวต์ที่สำคัญ ต่อแพ็กเก็ตฟิลเตอร์ริง ดังนี้

- ไอพีต้นทาง
- ไอพีปลายทาง
- ชนิดของโปรโตคอล (TCP UDP และ ICMP)

และในระดับทรานสปอร์ตเลเยอร์มีแอตทริบิวต์ที่สำคัญคือ

- พอร์ตต้นทาง
- พอร์ตปลายทาง
- แฟล็ก (Flag ซึ่งจะมีเฉพาะในเฮดเดอร์ของแพ็กเก็ต TCP)
- ชนิดของ ICMP message (ในแพ็กเก็ต ICMP)

ซึ่งพอร์ตของทรานสปอร์ตเลเยอร์ คือทั้ง TCP และ UDP นั้นจะเป็นสิ่งที่บอกถึงแอปพลิเคชันที่แพ็กเก็ตนั้นต้องการติดต่อด้วยเช่น พอร์ต 80 หมายถึง HTTP, พอร์ต 21 หมายถึง FTP เป็นต้น ดังนั้นเมื่อแพ็กเก็ตฟิลเตอร์ริง (Packet Filter) พิจารณาเฮดเดอร์ จึงทำให้สามารถควบคุมแพ็กเก็ตที่มาจากที่ต่าง ๆ และ มีลักษณะต่าง ๆ ได้

แพ็กเก็ตฟิลเตอร์ริง (Packet Filtering) สามารถอิมพลีเมนต์ได้ 2 แพลตฟอร์ม คือ

- เราเตอร์ที่มีความสามารถในการทำแพ็กเก็ตฟิลเตอร์ริง
- คอมพิวเตอร์ที่ทำหน้าที่เป็นเราเตอร์

ซึ่งจะมีข้อได้เปรียบเสียเปรียบกันดังนี้

	ข้อดี	ข้อเสีย
เราเตอร์	ประสิทธิภาพสูงมีจำนวน อินเตอร์เฟซมาก	เพิ่มเติมฟังก์ชันการทำงานได้ยาก อาจต้องการหน่วยความจำมาก
คอมพิวเตอร์ที่ทำหน้าที่เป็น เราเตอร์	เพิ่มฟังก์ชันการทำงานได้ไม่จำกัด	ประสิทธิภาพปานกลาง, จำนวน อินเตอร์เฟซน้อย , อาจมีความเสี่ยง จากระบบปฏิบัติการที่ใช้

ตารางที่ 3-1 เปรียบเทียบข้อดีข้อเสียในการเลือกอุปกรณ์มาทำหน้าที่แพ็กเก็ตฟิลเตอร์ริง

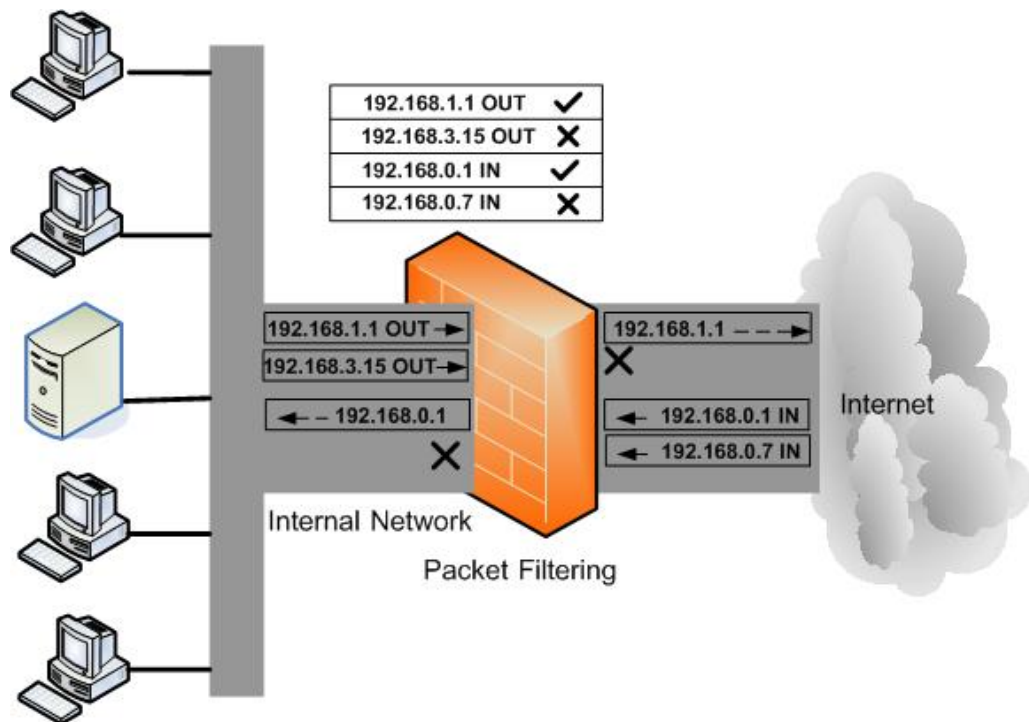
ข้อดี-ข้อเสียของแพ็กเก็ตฟิลเตอร์ริง

ข้อดี

- ไม่ขึ้นกับแอปพลิเคชัน
- มีความเร็วสูง
- รองรับการขยายตัวได้ดี

ข้อเสีย

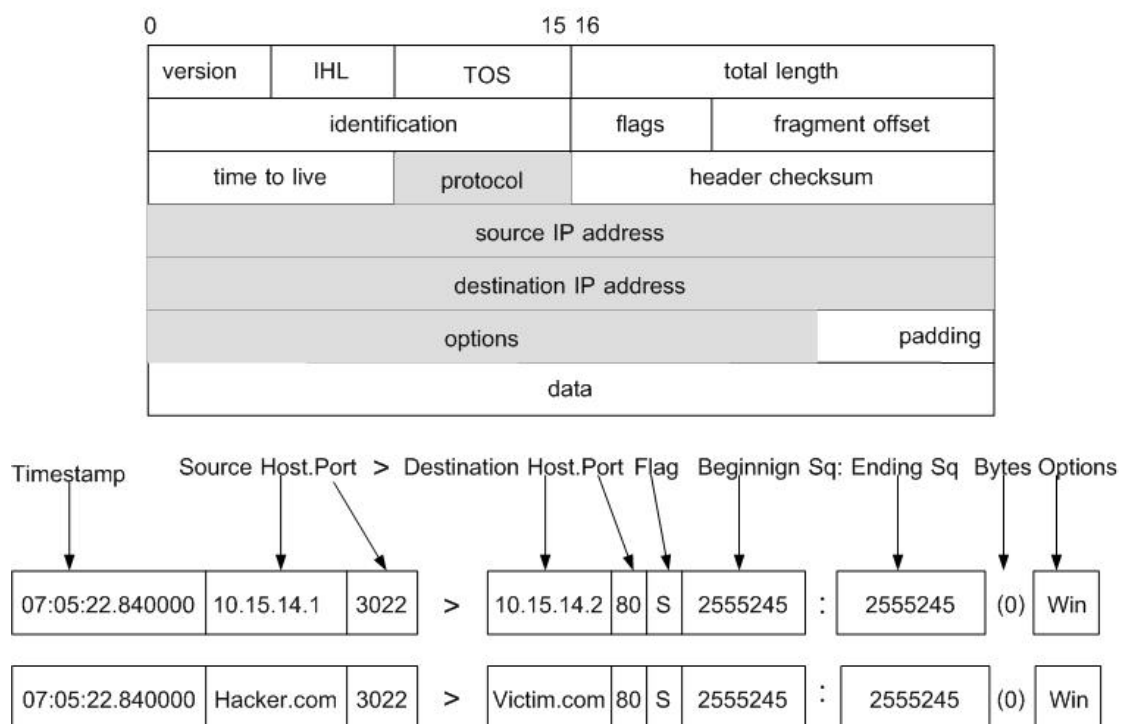
- บางโปรโตคอลไม่เหมาะสมกับการใช้ แพ็กเก็ตฟิลเตอร์ริง (Packet Filtering) เช่น FTP, ICQ



รูปที่ 3-1 แสดงการทำหน้าที่ของแพ็กเก็ตฟิเตอร์ริง (Packet Filtering)

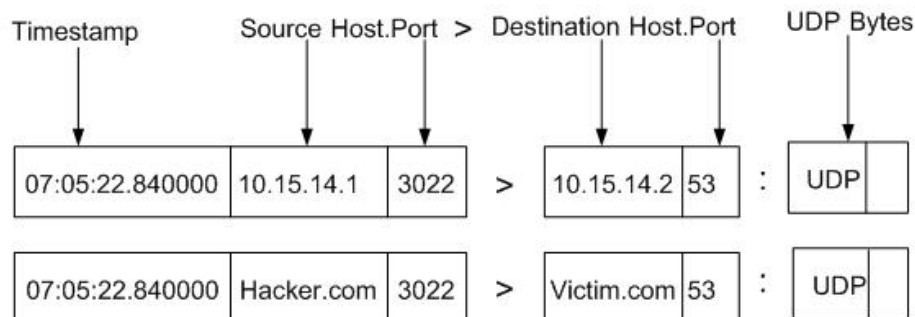
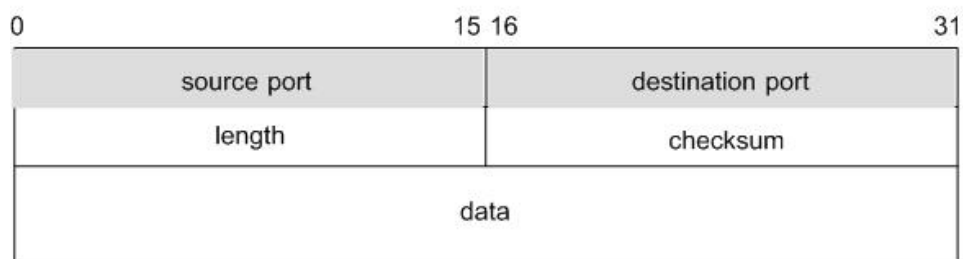
3.1.1.1 วิธีการอ่านแพ็กเก็ตเพื่อนำมาใช้ฟิเตอร์

1. โปรโตคอล ทีซีพี/ไอพี (TCP/IP)



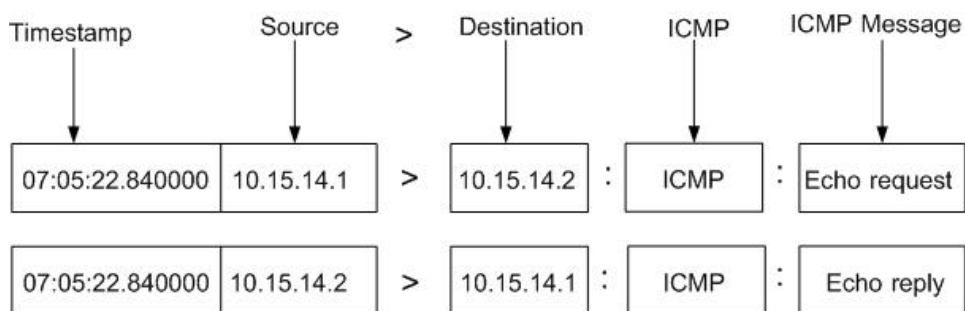
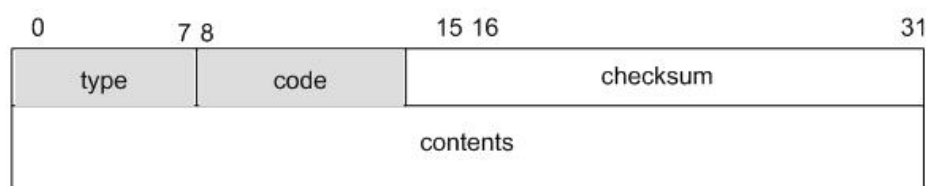
รูปที่ 3-2 แสดงการวิเคราะห์ฟิลด์ต่างๆ ของโปรโตคอล ทีซีพี (TCP)

2. โพรโทคอล ยูดีพี (UDP)



รูปที่ 3-3 แสดงการวิเคราะห์ฟิลด์ต่าง ๆ ของโปรโตคอล ยูดีพี (UDP)

3. โพรโทคอล ไอซีเอ็มพี (ICMP)



รูปที่ 3-4 แสดงการวิเคราะห์ฟิลด์ต่าง ๆ ของโปรโตคอล ไอซีเอ็มพี (ICMP)

3.1.2 พร็อกซี (Proxy)

พร็อกซี (Proxy) หรือ แอปพลิเคชันเกตเวย์ (Application Gateway) เป็นแอปพลิเคชันโปรแกรมที่ทำงานอยู่บนไฟร์วอลล์ ที่ตั้งอยู่ระหว่างเน็ตเวิร์ค 2 เน็ตเวิร์ค ทำหน้าที่เพิ่มความปลอดภัยของระบบเน็ตเวิร์คโดยการควบคุมการเชื่อมต่อระหว่างเน็ตเวิร์คภายใน กับเน็ตเวิร์คภายนอก พร็อกซี จะช่วย

เพิ่มความปลอดภัยได้มากเนื่องจากการตรวจสอบข้อมูลถึงในระดับของแอปพลิเคชันเลเยอร์ (Application Layer) เมื่อไคลเอนต์ (Client) ต้องการใช้เซิร์ฟเวอร์ภายนอก ไคลเอนต์จะทำการติดต่อไปยังพร็อกซีก่อน ไคลเอนต์จะเจรจา (negotiate) กับ พร็อกซี ติดต่อไปยังเครื่องปลายทางให้แล้วจะมีการเชื่อมต่อ (connection) 2 การเชื่อมต่อ คือ ไคลเอนต์ กับ พร็อกซี และ พร็อกซี กับเครื่องปลายทาง โดยที่พร็อกซี จะทำหน้าที่รับข้อมูล และ ส่งข้อมูลให้ใน 2 ทิศทาง ทั้งนี้ พร็อกซี จะทำหน้าที่ในการตัดสินใจว่าจะให้มีการเชื่อมต่อกันหรือไม่ จะส่งต่อแพ็กเก็ตให้หรือไม่

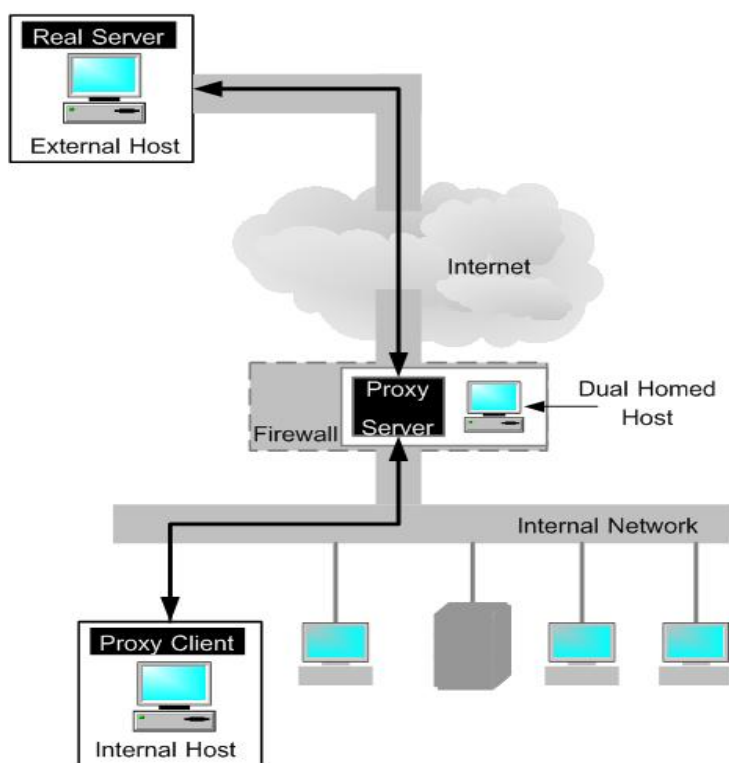
ข้อดี-ข้อเสียของพร็อกซี

ข้อดี

- มีความปลอดภัยสูง
- รู้จักข้อมูลในระดับแอปพลิเคชัน

ข้อเสีย

- ประสิทธิภาพต่ำ
- แต่ละบริการมักจะต้องการโปรเซสของตนเอง
- สามารถขยายตัวได้ยาก



รูปที่ 3-5 แสดงการใช้ Dual-homed Host เป็น พร็อกซีเซิร์ฟเวอร์ (Proxy Server)

3.1.3 สเตตฟูลอินสเปกชัน (Stateful Inspection)

โดยปกติแล้ว แพ็กเก็ตฟิเตอร์ริง (Packet Filtering) แบบธรรมดา จะควบคุมการเข้าออกของแพ็กเก็ตโดยพิจารณาข้อมูลจากเฮดเดอร์ของแต่ละแพ็กเก็ต นำมาเปรียบเทียบกับกฎที่มีอยู่ ซึ่งกฎที่มีอยู่ก็จะเป็นกฎที่สร้างจากข้อมูลส่วนที่อยู่ในเฮดเดอร์เท่านั้น ดังนั้นแพ็กเก็ตฟิเตอร์ริง แบบธรรมดาจึงไม่สามารถทราบได้ว่า แพ็กเก็ตนี้อยู่ส่วนใดของการเชื่อมต่อ เป็นแพ็กเก็ตที่เข้ามาติดต่อใหม่หรือเปล่า หรือว่าเป็นแพ็กเก็ตที่เป็นส่วนของการเชื่อมต่อที่เกิดขึ้นแล้ว เป็นต้น

สเตตฟูลอินสเปกชัน (Stateful Inspection) เป็นเทคโนโลยีที่เพิ่มเข้าไปใน แพ็กเก็ตฟิเตอร์ริง โดยในการพิจารณาว่าจะยอมให้แพ็กเก็ตผ่านไปนั้น แทนที่จะดูข้อมูลจากเฮดเดอร์เพียงอย่างเดียว สเตตฟูลอินสเปกชัน ยังจะนำเอาส่วนข้อมูลของแพ็กเก็ต (message content) และข้อมูลที่ได้จากแพ็กเก็ตก่อนหน้านี้ที่ได้ทำการบันทึกเอาไว้ นำมาพิจารณาด้วย จึงทำให้สามารถระบุได้ว่าแพ็กเก็ตใดเป็นแพ็กเก็ตที่ติดต่อเข้ามาใหม่ หรือว่าเป็นส่วนหนึ่งของการเชื่อมต่อที่มีอยู่แล้ว

	Packet Filtering	Stateful Inspection	Proxy
ข้อดี	● ประสิทธิภาพดี ● ง่ายในการ implement ● ไม่ขึ้นกับแอปพลิเคชัน	● ประสิทธิภาพดี ● เปิดพอร์ตเฉพาะเมื่อมีการติดต่อ ● สนับสนุนเกือบทุกบริการ	● ไม่เปิดเผยหมายเลขไอพีภายใน ● พิจารณาเนื้อหาของข้อมูลด้วย ● มี User Authentication ● เก็บรายละเอียด log
ข้อเสีย	● เปิดหมายเลขไอพีภายใน ● มีการเปิดช่องว่างทิ้งไว้ถาวร ● No User Authentication ● ใช้การเชื่อมต่อโดยตรงกับภายนอก	● No User Authentication ● ใช้การเชื่อมต่อโดยตรงกับภายนอก ● เปิดหมายเลขไอพีภายใน	● ประสิทธิภาพต่ำกว่า ● ต้องมีพรีอ็อกซีสำหรับทุก ๆ แอปพลิเคชันที่ใช้ ● ไม่มีการป้องกันที่ต่ำกว่าชั้นแอปพลิเคชัน ● เปิดเผยระบบปฏิบัติการ

ตารางที่ 3-2 เปรียบเทียบการทำงานของไฟร์วอลล์ทั้ง 3 ประเภท

3.2 กัยจากการโจมตี และ ความสามารถของไฟร์วอลล์

3.2.1 SYN Flooding

เครื่องผู้บุกรุกจะส่ง SYN Flag มายังเครื่องเป้าหมาย เพื่อทำ 3way-handshaking เมื่อเครื่องเป้าหมายได้รับ SYN Flag จะส่ง ACK และ SYN กลับไปยังเครื่องผู้บุกรุก ในขณะเดียวกันเครื่องเป้าหมายจะจองพื้นที่ในหน่วยความจำเพื่อรอรับ ACK ซึ่งหากผู้บุกรุกปลอมหมายเลขไอพี และ ส่ง SYN

มาเป็นจำนวนมาก หน่วยความจำของเครื่องเป้าหมายก็จะถูกใช้จนหมดไป จนเครื่องเป้าหมายไม่สามารถให้บริการได้หรือ ใช้งานไม่ได้ชั่วขณะ

SYN flooding เป็นการโจมตีในเชิงปริมาณ และความเร็ว ซึ่งกฎของไฟร์วอลล์ไม่ครอบคลุมถึงปริมาณ และ ความเร็วของข้อมูล การมีไฟร์วอลล์เป็นด่านหน้าจะช่วยบรรเทาความรุนแรงเนื่องจากการ SYN flood ได้ เพราะไฟร์วอลล์จะเป็นผู้รักษาการติดต่อไว้ทั้งหมดตั้งแต่เริ่ม SYN จนถึงสิ้นสุดเท่านั้น

3.2.2 PING Attack

ไอซีเอ็มพีมีหน้าที่ส่งข่าวสารและคำสั่งควบคุมของไอพี โดยเฉพาะการรายงานข้อผิดพลาดในการรับส่ง PING เป็นข้อความไอซีเอ็มพีประเภทหนึ่ง ใช้ในการหาข้อมูลเกี่ยวกับเครือข่าย

ซึ่งในการป้องกันโดยใช้ไฟร์วอลล์ไม่ควรให้เครื่องภายนอก หาข้อมูลเครื่องภายในเครือข่ายได้ โดยกำหนดกฎให้ เหมาะสม เช่นอาจจะไม่ให้มีการตอบสนอง ICMP packet เป็นต้น หรืออาจจะใช้โปรแกรมอื่นช่วยในการตรวจสอบการทำงานที่ผิดปกติ เช่น ระบบตรวจจับผู้บุกรุก (Intrusion Detection)

3.2.3 Tiny Fragment

หากแพ็กเก็ตมีขนาดใหญ่เกินกว่าที่ชั้นดาต้าลิงค์อนุญาต โปรโตคอลไอพี มีความสามารถในการแบ่งแพ็กเก็ตออกเป็นแพ็กเก็ตย่อย ๆ แล้วประกอบกลับให้เหมือนเดิมได้เมื่อแพ็กเก็ตถึงปลายทาง เนื่องจากโปรโตคอล ทีซีพี จะมีเฮดเดอร์แพ็กเก็ตของตัวเอง และถูกเติมเข้ากับเฮดเดอร์ของโปรโตคอลไอพีก่อนส่งไปยังปลายทาง หากแพ็กเก็ตมีขนาดใหญ่ แพ็กเก็ตก็จะถูกแบ่งออกเป็นแพ็กเก็ตย่อย ซึ่งจะมีเพียงแพ็กเก็ตแรกเท่านั้นที่บอกว่าแพ็กเก็ตนี้ถูกส่งมาจากหมายเลขไอพีใด หากเป็นหมายเลขไอพีที่ไม่ได้รับอนุญาตให้ผ่าน ไฟร์วอลล์จะมีเพียงแพ็กเก็ตแรกแพ็กเก็ตเดียวเท่านั้นที่ถูกครีโปก หรือ อาจจะไม่มีการครีโปกแพ็กเก็ตใด ๆ เลย หาก Sequence Number มีการแก้ไขเป็น 1 (ซึ่งปกติเป็น 0) เหมือนว่าไม่ใช่แพ็กเก็ตแรก

3.2.4 Port Scanning

การสแกนพอร์ตทุกครั้ง ผลลัพธ์ที่ได้คือรายละเอียดที่จะบอกได้ว่า โฮสต์ที่ถูกสแกนใช้ระบบปฏิบัติการอะไร มีแอปพลิเคชันใดทำงานอยู่บ้าง เปรียบเสมือนบอกผู้บุกรุกว่ามีช่องทางใดที่จะเจาะระบบเข้ามาได้

การสแกนพอร์ตจะทำให้ผู้บุกรุกทราบได้ว่ามีแอปพลิเคชันใดทำงานอยู่ หลังจากนั้นจะทำการสแกนแบบเจาะลึก ก็จะทราบว่าแอปพลิเคชันนั้นเป็นโปรแกรมอะไร เวอร์ชันใด มีข้อบกพร่องอย่างไร และควรนำเครื่องมือ หรือ เทคนิคใดในการเจาะ

ซึ่งการป้องกันโดยใช้ไฟร์วอลล์อาจจะต้องใช้โปรแกรมอื่นใช้งานร่วมเพื่อให้เกิดประสิทธิภาพการทำงานได้อย่าง ถูกต้องมาก

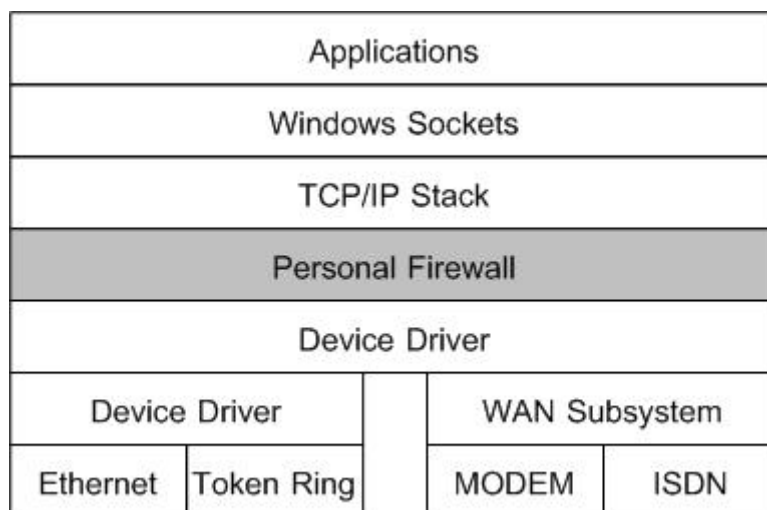
3.3 เพอร์ซันนอลไฟร์วอลล์ (Personal Firewall)

เพอร์ซันนอลไฟร์วอลล์จะมีความแตกต่างกับไฟร์วอลล์ทั่วไป เพราะไฟร์วอลล์ทั่วไปนั้นมีเป้าหมายหลักอยู่ที่ความปลอดภัยของเน็ตเวิร์คและ การควบคุมทราฟฟิกที่ผ่านเข้าออกระหว่าง เน็ตเวิร์คภายในกับ เน็ตเวิร์คภายนอก ทำงานอยู่ระหว่างโปรโตคอลในชั้น ทรานสปอร์ตเลเยอร์ (Transport Layer) กับ แอปพลิเคชันเลเยอร์ (Application Layer) แต่สำหรับเพอร์ซันนอลไฟร์วอลล์แล้ว การทำหน้าที่เพียงควบคุมทราฟฟิกเข้าออกนั้นไม่เพียงพอที่จะคุ้มครองป้องกันผู้ใช้ให้ได้อย่างปลอดภัย เพราะภัยของเครื่องคอมพิวเตอร์ส่วนบุคคลนั้นได้ครอบคลุมไปในหลายเรื่อง และส่วนใหญ่อีกที่มีความจำเป็นก็มีความจำเป็นที่จะต้องไปยุ่งเกี่ยวกับแอปพลิเคชันด้วย เดิมนั้นเพอร์ซันนอลไฟร์วอลล์มีรากฐานมาจากไฟร์วอลล์ปกติ เพียงแต่เป็นการนำมารวมเข้าไว้ในเครื่องของผู้ใช้ เสียเลยแทนที่ จะต้องแยกเป็นไฟร์วอลล์ต่างหากอีกเครื่องหนึ่งซึ่งจะเป็นการสิ้นเปลืองโดยใช่เหตุ แต่ในเมื่อภัยหลายประการที่จะส่งผลกระทบต่อผู้ใช้ ผู้ผลิตจึงได้นำโปรแกรมป้องกันหลาย ๆ ชนิดมารวมกันเป็นชุดโปรแกรมรักษาความปลอดภัยสำหรับเครื่องคอมพิวเตอร์ส่วนบุคคลโดยประกอบ ด้วย

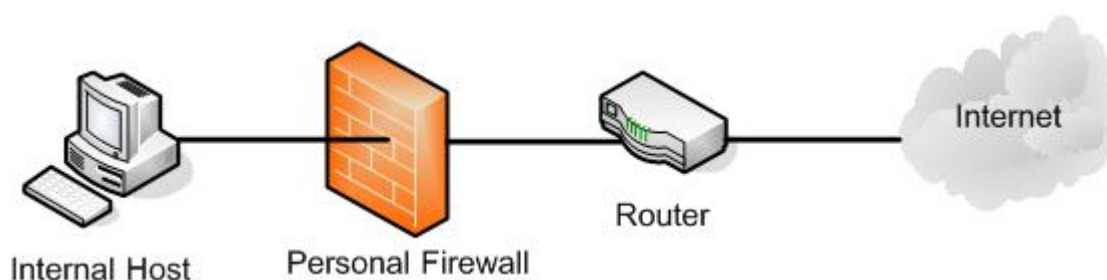
- ไฟร์วอลล์ (Firewall) สำหรับควบคุมทราฟฟิก เป็นการป้องกันการเจาะระบบของแฮกเกอร์
- โปรแกรมป้องกันการทำงานของแอดทีฟคอนเทนต์
- โปรแกรมป้องกันเว็บไซต์ที่ไม่เหมาะสม (Parental Control)
- โปรแกรมบล็อกโฆษณาทางอินเทอร์เน็ต (Advertising Blocking)
- โปรแกรมป้องกันไวรัส, ตรวจสอบหาโปรแกรมม้าโทรจัน และ โปรแกรมประเภทแบ็คดอร์
- โปรแกรมตรวจจับ และ ป้องกันการบุกรุก (Intrusion Detection & Protection)

ซึ่งขึ้นอยู่กับบริษัทผู้ผลิตว่าจะรวบรวมโปรแกรมเหล่านี้เข้าด้วยกันมากน้อยแค่ไหน อย่างไรก็ตาม เพอร์ซันนอลไฟร์วอลล์ที่มีจำหน่ายอยู่ในปัจจุบันจึงกลายเป็นชุดรักษาความปลอดภัยแบบครบวงจร

ในหน้าที่ส่วนที่เป็นไฟร์วอลล์นั้นนอกจากจะมีขอบเขตหน้าที่การทำงานที่ต่างจาก ไฟร์วอลล์ ทั่วไป แล้ว โครงสร้างในทางเทคนิคของเพอร์ซันนอลไฟร์วอลล์ก็มีความแตกต่างจากไฟร์วอลล์ธรรมดาอยู่มากพอสมควร กล่าวคือ ไฟร์วอลล์ทั่วไปมักจะมีโครงสร้างที่เป็นองค์ประกอบของฮาร์ดแวร์ และ ซอฟต์แวร์ ทำงานร่วมกัน มีเน็ตเวิร์คอะแดปเตอร์ คอยรับข้อมูลที่เข้ามาและส่งข้อมูลผ่านการตรวจสอบแล้วออกไป แต่สำหรับเพอร์ซันนอลไฟร์วอลล์นั้นเป็นเพียงซอฟต์แวร์เท่านั้น ไม่มีฮาร์ดแวร์ใด ๆ มาเกี่ยวข้อง แม้แต่น้อย เป็นซอฟต์แวร์ที่ทำหน้าที่ตรวจตราและควบคุมการเรียกใช้เน็ตเวิร์คดีไวซ์บนเครื่องคอมพิวเตอร์นั่นเอง



รูปที่ 3-6 เพอร์ซันนอลไฟร์วอลล์เมื่อรวมเข้ากับซอฟต์แวร์บนโฮสต์



รูปที่ 3-7 แสดงการนำไฟร์วอลล์มาให้บริการในลักษณะของเพอร์ซันนอลไฟร์วอลล์

3.4 Firewall API

ในการเขียนโปรแกรมบนวินโดวส์ เพื่อติดต่อและควบคุมลงไปถึง NIC (Network Interface Card) แล้วจะมีความยุ่งยากมาก เพราะ NIC แต่ละอันก็มีความแตกต่างกันไป โดยทั่ว ๆ ไปจะเป็นการนำ API (Application Program Interface) ที่เตรียมไว้ใน Microsoft SDK (Software Development Kit) มาใช้ในการพัฒนาโปรแกรม จะทำให้การพัฒนาโปรแกรมบนวินโดวส์ทำได้ง่ายขึ้น

3.4.1 Firewall-Hook Driver

การพัฒนาไฟร์วอลล์บนระบบปฏิบัติการวินโดวส์ 2000 จะใช้วิธีฮุค (Hook) ไดรเวอร์ของไฟร์วอลล์เข้ากับระบบของระบบปฏิบัติการ โดยวิธีนี้ไม่ใคร่ซอฟต์แวร์ ไม่ได้ให้ข้อมูลเกี่ยวกับเรื่องนี้มากนัก ซึ่งมีทางเป็นไปได้ที่จะศึกษาคือ ต้องศึกษา DDK header file (Microsoft Driver Development Kit) ซึ่งก็คือไฟล์ ipFirewall.h ซึ่งเป็นวิธีการที่น่าสนใจมากเพราะเป็นการพัฒนาโปรแกรมประเภทแพ็คเก็ตฟิลเตอร์ริง (Packet Filtering) ได้อย่างมีประสิทธิภาพ

3.4.2 การฟิลเตอร์แพ็กเก็ตบนวินโดวส์

สำหรับสถาปัตยกรรมของ NDIS นั้นได้อนุญาตให้สามารถที่จะเขียนโปรแกรมเพื่อเข้าไปฟิลเตอร์แพ็กเก็ตได้หลายวิธีโดยการเรียกใช้ฟังก์ชันการทำงานจากชั้นต่าง ๆ ของ NDIS ซึ่งแต่ละวิธีก็มีวิธีการ รูปแบบการใช้งาน และความสามารถในการทำงานที่แตกต่างกันไป มีทั้งในแบบ User-Mode และ Kernel-Mode

- User-Mode Network Data Filtering

- Winsock Layered Service Provider (LSP) เป็นไปได้ที่เราสามารถที่จะเรียก Kernel-Mode TCP/IP Driver ผ่าน Transport Data Interface (TDI) โดยไม่จำเป็นต้องผ่าน Winsock แต่ในงานที่จำเป็นจะต้อง ตรวจสอบหรือกระทำกับทุก ๆ แพ็กเก็ตนั้นไม่ควรที่จะอิงการทำงานของ Winsock LSP แต่ควรจะประยุกต์ใช้วิธีการ Kernel-mode มากกว่า
- Windows 2000 Packet Filtering Interface สำหรับระบบปฏิบัติการวินโดวส์ 2000 ได้มี API (Application Program Interface) ที่อนุญาตให้แอปพลิเคชัน หรือ เซอร์วิสที่ทำงานใน User-mode สามารถที่จะกำหนดชุดของ Filter Descriptor ซึ่งจะถูกนำไปใช้โดยคอมโพเนนต์ (Component) ที่ซีพี/ไอพี (TCP/IP) ในชั้นล่าง การฟิลเตอร์นี้จะดูที่ หมายเลขไอพีต้นทาง หมายเลขไอพีปลายทาง และ หมายเลขพอร์ต โดยสามารถกำหนดได้เพียงแค่อนุญาตให้ผ่าน หรือ ไม่ให้ผ่าน เท่านั้น
- Winsock Replacement DLL ก่อนที่จะมี LSP วิธีการเดียวที่จะเพิ่มความสามารถของฟังก์ชันในการทำงานของ Winsock ก็คือการสร้าง DLL ชุดใหม่มาทับ DLL ของ Winsock ซึ่งก็สามารถประยุกต์ใช้ในการฟิลเตอร์ แพ็กเก็ตได้ แต่ไม่เป็นที่นิยมเนื่องจากความยากในการทำ อีกทั้งยังต้องยุ่งเกี่ยวกับฟังก์ชันการทำงานที่ไม่มีคู่มืออธิบายอีกด้วย

- Kernel-Mode Network Data Filtering

- Transport Data Interface (TDI) Filter Driver ทำงานในชั้นของ TCP/IP Driver
- NDIS Intermediate Driver ทำงานอยู่ระหว่าง Miniport Driver กับ Transport Driver ซึ่งสามารถที่จะประยุกต์ฟิลเตอร์แพ็กเก็ตในชั้นนี้ได้เช่นกัน
- Windows 2000 Filter-Hook Driver เป็นตัวที่ ขยายขอบข่ายความสามารถออกมาจาก IP Filter Driver ซึ่งมากับระบบปฏิบัติการ
- NDIS Hooking Filter Driver

3.4.3 การทำงานของไฟร์วอลล์แบบ Hook Driver ที่สร้างขึ้น

ไคร์เวอร์ของไฟร์วอลล์แบบ ฮุก นี้ไม่ใช่ไคร์เวอร์ของเน็ตเวิร์ค ดังนั้นมันจะทำงานในโหมดของเคอร์เนล (Kernel Mode) โดยการติดตั้งฟังก์ชันคอลแบ็ก (Callback Function) โดยเริ่มที่ติดตั้งไคร์เวอร์ไฟร์วอลล์แบบ ฮุก นี้โดยไฟล์ ipFirewall.h จะมีดังนี้

```

typedef struct _IP_SET_FIREWALL_HOOK_INFO
{
    // Packet filter callout.
    IPPacketFirewallPtr FirewallPtr;

    // Priority of the hook
    UINT Priority;

    // if TRUE then ADD else DELETE
    BOOLEAN Add;
} IP_SET_FIREWALL_HOOK_INFO, *PIP_SET_FIREWALL_HOOK_INFO;

#define DD_IP_DEVICE_NAME L "\\Device\\Ip"
#define _IP_CTL_CODE (function, method, access) \
    CTL_CODE (FSCTL_IP_BASE, function, method, access)
#define IOCTL_IP_SET_FIREWALL_HOOK \
    _IP_CTL_CODE (12, METHOD_BUFFERED, FILE_WRITE_ACCESS)

```

ซึ่งเราสามารถกำหนดค่าต่างๆ ให้กับโครงสร้าง โดยในฟิลด์ Priority เป็นส่วนที่กำหนดลำดับของฟังก์ชันฟิลเตอร์ และเราสามารถนำไปใช้ดังนี้

```

PDEVICE_OBJECT ipDeviceObject=NULL;
IP_SET_FIREWALL_HOOK_INFO filterData;

//.....

// Init structure filterData.
FirewallPtr = filterFunction;
filterData.Priority = 1;
filterData.Add = TRUE;

//.....

// Send the commando to ip driver
IoCallDriver(ipDeviceObject, irp);

```

ถ้าต้องการเลิกติดตั้งฟังก์ชันฟิลเตอร์ สามารถทำได้โดยใช้โค้ดเดิม แต่ใส่ค่าที่ filterData.Add เป็น FALSE

ฟังก์ชันฟิลเตอร์ (The Filter Function)

ฟังก์ชันฟิลเตอร์สำหรับไฟร์วอลล์แบบฮุกนี้ จะมีความซับซ้อนมาก เพราะไม่มีข้อมูลเกี่ยวกับฟังก์ชันและพารามิเตอร์ โดยฟังก์ชันฟิลเตอร์มีดังนี้

```
FORWARD_ACTION cbFilterFunction(VOID**pData,
    UINT RecvInterfaceIndex,
    UINT* pSendInterfaceIndex,
    UCHAR* pDestinationType,
    VOID* pContext,
    UINT ContextLength,
    Struct IPRecvBuf**pRcvBuf);
```

พารามิเตอร์ต่างๆ มีดังนี้

pData * pData เป็นพอยเตอร์ (pointer) ที่ชี้ไปยังโครงสร้างที่เก็บข้อมูลแพ็กเก็ต (struct IPRecvBuf*)

RecvInterfaceIndex เป็นส่วนที่รับข้อมูลเข้ามา

pSendInterfaceIndex เป็นพอยเตอร์ (pointer) ชนิด unsigned int เก็บค่าที่ชี้ข้อมูลที่ส่ง

pDestinationType เป็นพอยเตอร์ (pointer) ชนิด unsigned int เก็บค่าของเป้าหมาย เช่น network, remote, broadcast, multicast, etc.

pContext เป็นพอยเตอร์แบบโครงสร้าง FIREWALL_CONTEXT_T เพื่อหาข้อมูลของแพ็กเก็ตที่เข้าหรือออก

pRcvBuf*pRcvBuf ถูกเซตให้เป็น NULL เสมอ

ข้อมูลทั้งหมดนี้อาจมีการเปลี่ยนแปลงได้ในวินโดวส์เวอร์ชันต่อไปในอนาคต เพราะไม่มีข้อมูลอย่างเป็นทางการ แต่ข้อมูลเหล่านี้ได้ถูกทดสอบในวินโดวส์ 2000 และวินโดวส์ XP

แต่ละแพ็กเก็ต เมื่อฟังก์ชันถูกเรียกใช้ การทำงานจะขึ้นอยู่กับค่าที่ส่งกลับมาว่าจะให้แพ็กเก็ตผ่านหรือไม่ให้ผ่าน ซึ่งค่าที่ส่งกลับมีดังนี้

- FORWARD แพ็กเก็ตจะถูกอนุญาตให้ผ่าน
- DROP แพ็กเก็ตจะถูกปฏิเสธไม่ให้ผ่าน
- ICMP_ON_DROP แพ็กเก็ตจะถูกปฏิเสธไม่ให้ผ่านและแพ็กเก็ต ICMP จะถูกส่งไปที่เครื่องรีโมทโดยในตอนนี้ซึ่งสามารถกำหนดได้ทั้งการฟิลเตอร์ทั้งขาเข้าและขาออก พารามิเตอร์หลักสำหรับฟังก์ชันนี้ คือ PF_FILTER_DESCRIPTOR เพื่อเป็นข้อกำหนดกฎต่างๆ ในฟังก์ชันการฟิลเตอร์ ที่จะส่งเข้าไปยังไดร์เวอร์ไฟร์วอลล์แบบฮุกให้ทำการฟิลเตอร์นั่นเอง

ฟังก์ชัน PF_FILTER_DESCRIPTOR

Filter Descriptor นั้นเป็นรูปแบบโครงสร้างของกฎการฟิเตอร์ ในการที่เราจะกำหนดกฎแต่ละข้อซึ่งในขั้นตอนแรกนั้น เราจะต้องสร้าง Filter Descriptor ขึ้นมาและกำหนดค่าต่างๆ โดยโครงสร้างของ PF_FILTER_DESCRIPTOR นั้นเป็นดังต่อไปนี้

```

Typedef struct PF_FILTER_DESCRIPTOR {
    DWORD dwFilterFlage;

    DWORD dwRule;

    PFADDRESSSTYPE pfatType;

    PBYTE SrcAddr;

    PBYTE SrcMadk;

    PBYTE DstSddr;

    PBYTE DdtMask;

    DWORD dwProtocol;

    DWORD fLateBound;

    WORD wSrcPort;

    WORD wDstpor;

    WORD wSrcportHighRange;

    WORD wDstPortHighRange;

} PF_FILTER_DESCRIPTOR, *PPF_FILTER_DESCRIPTOR;

```

โดยมีรายละเอียดของแต่ละฟิลด์ดังนี้

dwFilterFlags ณ ขณะนี้มีเพียงแค่ค่าเดียวที่ถูกกำหนดขึ้นสำหรับฟิลด์นี้ คือ

FD_FLAGS_NOSYN โดยเมื่อแอปพลิเคชันต้องการที่จะสร้างการเชื่อมต่อแบบ TCP จะต้องมีการส่ง synchronization request ซึ่ง Flag นี้เป็นการห้ามไม่ให้ทำเช่นนั้น แต่จากการทดลองพบว่าไม่มีความแตกต่างในการกำหนดหรือไม่กำหนดค่าให้สำหรับ flag นี้

dwRule เป็นค่าที่ผู้ใช้กำหนดขึ้นซึ่งจะถูกส่งไปสู่ log (ถ้ามีการใช้งาน log) เพื่อแสดงว่ากฎการฟิเตอร์ข้อไหนที่รับผิดชอบสำหรับแต่ละข้อของ log

pfatType กำหนดเวอร์ชันของไอพี ซึ่งสามารถกำหนดได้เป็น PF_IPV4 (IP เวอร์ชัน 4)หรือ PF_IPV6 (IP เวอร์ชัน 6)

SrcAddr หมายเลขไอพีต้นทาง

SrcMask เน็ตมาสก์ต้นทาง

DstAddr หมายเลขไอพีปลายทาง

DstMask เน็ตมาสก์ปลายทาง

หมายเลขไอพีต้นทาง ปลายทาง รวมถึงมาสก์ เป็นการกำหนดช่วงของหมายเลขไอพีที่กฎการฟิลเตอร์มีผล โดยมาสก์เป็นตัวกำหนดขอบเขตของแอดเดรสที่จะถูกทดสอบ และเมื่อแอดเดรสถูกกำหนดเป็นช่วง (โดยการกำหนดส่วนของมาสก์เป็นศูนย์) ส่วนของแอดเดรสที่สัมพันธ์กันก็ต้องถูกกำหนดให้เป็นศูนย์ด้วยเช่นกัน ตัวอย่างเช่น ถ้าต้องการกำหนดหมายเลขไอพีหมายเลขเดียวแอดเดรสจะต้องสมบูรณ์และค่ามาสก์จะต้องเป็น FF.FF.FF.FF. ในทางกลับกัน ถ้าต้องการจะกำหนดฟิลเตอร์สำหรับช่วงของหมายเลขไอพี ก็ต้องกำหนดส่วนที่เป็นช่วงของแอดเดรส ให้เป็นศูนย์เช่นเดียวกันกับส่วนเดียวกันสำหรับมาสก์ (แอดเดรส a.b.c.0 และมาสก์ FF.FF.FF.00 จะฟิลเตอร์หมายเลขไอพีทั้งหมดที่ขึ้นต้นด้วย a.b.c) ในการกำหนดฟิลเตอร์ขาออก จะต้องกำหนดให้แอดเดรสของตัวเองเป็นแอดเดรสต้นทาง ส่วนการฟิลเตอร์ขาเข้าจะต้องกำหนดแอดเดรสของฝั่งรีโมท (Remote)เป็นต้นทาง dwProtocol กำหนดโปรโตคอลไอพีที่จะต้องฟิลเตอร์ ซึ่งมี ICMP,TCP,UDP หรือทั้งหมด fLateBound ใช้ในการกำหนดข้อมูลเกี่ยวกับแอดเดรสที่ต้องการถูกปรับปรุงเมื่อมีการ rebound ฟิลเตอร์ (เช่นเมื่อมีการเชื่อมต่อใหม่ของ dial-up adapter) wSrcPort หมายเลขพอร์ตต้นทาง wSrcPortHighRange ขอบบนของหมายเลขพอร์ตต้นทาง wSrcport หมายเลขพอร์ตปลายทาง wDstportHighRange ขอบบนของหมายเลขพอร์ตปลายทาง การกำหนดพอร์ตในการฟิลเตอร์นั้น ถ้ากำหนดหมายเลขพอร์ตเป็นศูนย์จะหมายถึงทุกพอร์ต แต่ถ้ากำหนดหมายเลขพอร์ตที่ไม่ใช่ศูนย์แล้วจะต้องกำหนด PortHighRange ด้วยโดยถ้าต้องการระบุพอร์ตใดพอร์ตหนึ่งก็ให้กำหนดทั้งสองฟิลด์ให้เป็นค่าเดียวกัน แต่ถ้าต้องการกำหนดเป็นช่วงพอร์ตก็ให้กำหนด PortHighRange ให้เป็นค่าช่วงของหมายเลขพอร์ต กล่าวคือ กำหนดค่าพอร์ตให้เป็นหมายเลขพอร์ตต่ำสุดที่ต้องการฟิลเตอร์ และ PortHighRange เป็นค่าสูงสุดของพอร์ตที่ต้องการฟิลเตอร์ เช่น ถ้าต้องการให้มีการฟิลเตอร์ตั้งแต่ พอร์ต 21 ถึงพอร์ต 48 ก็ให้กำหนด Port เป็น 21 และ PortHighRange เป็น 48 เป็นต้น

บัฟเฟอร์ของแพ็กเก็ตในไคร์เวอร์ของไฟร์วอลล์แบบฮาร์ดแวร์ จะไม่ได้รับส่วนหัวของแพ็กเก็ตและส่วนที่บรรจุในแพ็กเก็ตนั้นของบัฟเฟอร์โดยทันที แต่บัฟเฟอร์นั้นจะมีโครงสร้างการทำงาน ทั้งส่งและรับแพ็กเก็ตที่ผ่านในพารามิเตอร์ pData โดย *pData จะชี้ที่ IPRcvBuf ซึ่งมีโครงสร้างดังนี้

```
Struct IPRcvBuf
{
    //Point to the next buffer in the chain
    struct IPRcvBuf*ipr_next;
    //Always 0
    UINT ipr_owner;
    //Buffer data
    UCHAR *ipr_buffer;
```

```

//Buffer data size
UINT ipr_size;

//In my tests always a pointer to NULL.
//Maybe the system could use MDLs instead of IPRcvBuf structures
//(but i never have seen it).
PMDL_ipr_pMdl;

//Always a pointer to NULL.
UINT*ipr_pClientCnt;

//Always a pointer to NULL.
UCHAR*ipr_RcvContext;

//Always 0. I suppose this field is an offset into buffer data
//but because I haven't a value different from 0, I can affirm it.
UINT ipr_RcvOffset;

//In windows 2003 DDK the name of this field have changed to flags.
//In my tests I always get 0 value for local traffic and 0 for
//remote. It's the only thing I can tell you about this field.
ULONG ipr_promiscuous;
};

```

จากโครงสร้างนี้ ฟิวด์ ipr_buffer จะเก็บ ipr_size ซึ่งเป็นขนาดของแพ็กเก็ตมีหน่วยเป็นไบต์ ส่วน ฟิวด์ ipr_next จะใช้เก็บโครงสร้างต่อไป เพราะการเก็บข้อมูลของแพ็กเก็ตนี้จะเก็บเป็นโครงสร้างของข้อมูลต่อๆ กันไปจนโครงสร้างสุดท้ายจะชี้ไปที่ NULL โดยในแพ็กเก็ต ซึ่งมีลักษณะการใช้งานดังนี้

```

Char *pPacket=NULL;
Int iBufferSize;
Struct IPRcvBuf*pBuffer=(struct IPRcvBuf*)pData;

//First, I calculate the total size of the packet

```

```

iBufferSize=buffer->ipr_size;
while(pBuffer->ipr_next!=NULL)
{
    pBuffer=pBuffer->ipr_next;
    iBufferSize+=pBuffer->ipr_size;
}
//Reserve memory to the lineal buffer.
pPacket=(char*)ExAllocatePool(NonPagedPool, iBufferSize);
if(pPacket!=NULL)
{
    unsigned int iOffset = 0;
    pBuffer = (struct IPRcvBuf*)pData;

    //we are going to copy each buffer of the chain in the lineal buffer.
    memcpy(pPacket, pBuffer->ipr_buffer,pBuffer->ipr_size);
    While(pBuffer ->ipr_next!= NULL)
    {
        iOffset = pBuffer ->ipr_size;
        pBuffer = pBuffer->ipr->next;
        memcpy(pPacket + iOffset,pBuffer->ipr_buffer,
            pBuffer ->ipr_size);
    }
}

```